Contracts, Endpoints, and Handles

An introduction to Okapi's core vocabulary

Okapi

July 12, 2026

Contents

Introduction

Okapi describes an HTTP API as ordinary data before it describes how to run one. This document walks through that vocabulary from the ground up, in the order you'd actually build with it: a **request contract** and a **response contract**, combined into a **Contract**; a Contract combined with a **Function** into an **Endpoint**; the several ways to turn one or many Endpoints into a running server; and — the payoff for all of it — how the exact same Contract that describes your server also generates a type-safe client, type-safe links, and OpenAPI docs, with a compiler guarantee that none of them can drift out of sync with what the server actually serves.

Every example below is real, working Okapi code — nothing has been simplified past the point of actually type-checking, and every non-trivial one has been run, not just compiled.

```haskell
{-# LANGUAGE ApplicativeDo #-}
{-# LANGUAGE BlockArguments #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE DeriveAnyClass #-}
{-# LANGUAGE DeriveGeneric #-}
{-# LANGUAGE DuplicateRecordFields #-}
{-# LANGUAGE OverloadedRecordDot #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TypeApplications #-}

import Okapi
import Okapi.HTTP.Request.Body (json)
import Okapi.HTTP.Response qualified as Res
import Okapi.HTTP.Response.Headers qualified as ResH
import Okapi.Record.Data qualified as Data
import Okapi.Record.Tree (Request (..), Response (..))
import Okapi.Record.Tree qualified as Tree
import Data.Aeson qualified as Aeson
import Data.Aeson (ToJSON, FromJSON)
import Data.ByteString.Lazy qualified as LBS
import Data.Function ((&))
import Data.OpenApi (ToSchema, OpenApi)
import Data.Text (Text)
import GHC.Generics (Generic)
import Network.HTTP.Types qualified as HTTP
```

Every construction shown below has at least one alternative spelling further on — Okapi doesn't force one calling convention. Pick whichever reads best at the call site.

Request contracts

A request contract describes how to parse an incoming request — its method, path, query, headers, and body — as pure data, independent of any handler. `requestGET`/`requestPOST`/etc. are pre-built starting points; you customize only the fields your endpoint actually needs, using ordinary record update syntax and do-notation (an Applicative, not a Monad — hence `ApplicativeDo`) for multi-segment paths:

```haskell
getUserReq = requestGET
    { path = do
        segment_ text "users"
        uid <- segment "userId" int
        pure uid
    }
```

`getUserReq` reads a path like `/users/42`, discarding the literal `"users"` segment and keeping `42` as an `Int`.

Every field also has a same-named setter function, built for piping with `(&)`:

```haskell
getUserReq' = requestGET
    & path do
        segment_ text "users"
        uid <- segment "userId" int
        pure uid
```

Both `getUserReq` and `getUserReq'` are the same value.

Response contracts

A response contract is the same idea for the other side of the wire — a status, headers, and a body:

```
getUserRes = response200
```

`response200`, `response201`, `response404`, and friends are pre-built responses for the common cases; `response` is the fully generic form when you need an arbitrary status.

One request, one response: Contract via `(:->)`

A single request paired with a single response is a `Contract` — built by naming both pieces first, then combining them with `(:->)`:

```
getUserContract = getUserReq :-> getUserRes
```

One request, many responses: Contract via `(:-<)` and `cases`

Some endpoints don't have one response shape — creating a user might succeed or hit a conflict, each with its own status and body. Declare every alternative as one field of a sum type, and combine the request with the whole set of alternatives:

```
createUserReq = requestPOST
    { path = segment_ text "users"
    , body = json @NewUser
    }

data CreateUserResponses f
    = Created (f S201       HTTP.ResponseHeaders (IO LBS.ByteString))
    | Other   (f HTTP.Status HTTP.ResponseHeaders (IO LBS.ByteString))
    deriving (Generic, Cases)

createUserResponses = cases @CreateUserResponses
    response201
    response

createUserContract = createUserReq :-< createUserResponses
```

`S201` is short for `KnownStatus 201` — every recognized status code (`S100`, `S404`, `S500`, …) has one of these, mirroring `GET`/`POST`/etc. being short for `KnownMethod "GET"`/`KnownMethod "POST"` for methods. Both are purely type-level abbreviations; `S201` and `KnownStatus 201` are the same type, so a value built with one type-checks fine against a signature written with the other.

Shape helpers: `Origin` and `(:&)`

A `Contract`'s type is `Contract (Shape method path query headers body result)` — six type arguments capturing everything about the wire shape. Most of the time you never write this out; it's inferred from `getUserReq :-> getUserRes`. But once records enter the picture (the next section), a field's type has to be written explicitly — GHC can't infer a record field's type the way it infers a `let`-bound value's:

```
data Routes f = Routes
    { getUser :: f
        (Shape GET Int HTTP.Query HTTP.RequestHeaders (IO LBS.ByteString)
         (Data.Response S200 HTTP.ResponseHeaders (IO LBS.ByteString)))
    }
```

That's a lot of ceremony to say "a GET with a captured `Int`, and a plain 200 response." `Origin` and `(:&)` are an optional, purely type-level convenience for exactly this: `Origin` is the maximally unconstrained `Shape` — literally what `request :-> response` (the fully generic forms) already infers — and `(:&)` overrides one slot at a time, like a record update:

```
type UserShape =
    Origin
        :& METHOD GET
        :& PATH Int
        :& RESPOND (Data.Response S200 HTTP.ResponseHeaders (IO LBS.ByteString))

data Routes f = Routes
    { getUser :: f UserShape
    }
```

Only the slots that actually differ from "nothing specified" need naming. `METHOD/PATH/QUERY/HEADERS/BODY/RESPOND` are deliberately all-caps rather than the bare slot names — `Method/Path/Query/ Headers/Body` already name the Tree DSL's own types elsewhere in Okapi, and Haskell's case-sensitivity is what keeps the two from colliding.

If the response side should stay fully generic too, `AnyResponse` is a synonym for exactly that — `Data.Response HTTP.Status HTTP.ResponseHeaders (IO LBS.ByteString)`, i.e. `Origin`'s own result slot:

```
type UserShape = Origin :& METHOD GET :& PATH Int :& RESPOND AnyResponse
```

This is purely a type-level abbreviation — it doesn't change what a `Contract/Function/Endpoint` is, or how you build one; it only makes writing their types shorter. Skip it entirely if you'd rather spell `Shape` out — both are the same type, and everything downstream treats them identically.

Contract + Function = Endpoint

A Contract alone is just a description — nothing to run. Pair it with a **Function** (the actual handler, built with `fn`) and a natural transformation (`transform`, how to run the handler's monad down to `IO`), and — one more field, `middleware`, a `Wai.Middleware` scoped to just this one route (pass `id` for none) — you have an **Endpoint**:

```
getUserEndpoint = Endpoint
    { transform  = id
    , middleware = id
    , contract   = getUserContract
    , function   = fn \(reqVal, _waiReq) ->
        pure Data.Response
            { status  = 200
            , headers = []
            , body    = pure (Aeson.encode User { userId = reqVal.path, name = "Ada Lovelace" })
            }
    }
```

`Endpoint` also has a plain positional smart constructor, `endpoint`, for when record syntax feels heavier than the call site needs — argument order is `transform`, `middleware`, `contract`, `function`:

```
getUserEndpoint' = endpoint id id getUserContract $ fn \(reqVal, _waiReq) ->
    pure Data.Response
        { status  = 200
        , headers = []
        , body    = pure (Aeson.encode User { userId = reqVal.path, name = "Ada Lovelace" })
```

```
        }
```

Give one endpoint its own middleware — auth, logging, whatever's scoped to just this route — with `scope`, which composes onto whatever's already there:

```
authedGetUserEndpoint = scope requireAuthMiddleware getUserEndpoint
```

Everything an `Endpoint` needs to actually answer a request — what it parses, how it's run, what middleware wraps it, and what it does — now lives in one value.

Four ways to turn Endpoints into a server

Every one of these ends the same way: a `Wai.Middleware`, applied to a fallback (`catchAll`, Okapi's bundled 404, or your own) to get a runnable `Wai.Application`. They differ in how you get there, because "how many endpoints, organized how" isn't one problem — it's at least four.

1. One endpoint, directly

`route` dispatches a single `Endpoint`. Chain several with plain function composition — no new combinators to learn:

```
app = route getUserEndpoint
    . route createUserEndpoint
    $ catchAll
```

Right for a small, fixed, known-at-the-call-site set. Zero indirection — no list, no `Generic`, nothing beyond the `Endpoint`s you already built.

2. A list of endpoints — the foundational mechanism

`Handle` wraps an `Endpoint`, hiding its type variables so a list of them is an ordinary, homogeneous `[Handle]`, even across endpoints running under entirely different monads or answering entirely different request shapes:

```
handles' = [handle getUserEndpoint, handle createUserEndpoint]

app = run handles' catchAll
```

`run` folds a `[Handle]` into one `Wai.Middleware` via `mount` — the same currency `route` produces, so the two mix freely. This is the *only* mechanism with no `Generic` involved at all (a real compile-time win at scale — see the closing section), and the only one where the same collection serves two purposes: `run` for the app, `foldMap toOpenApi` for docs, since `Handle` retains the `Endpoint` rather than erasing straight to an opaque function.

3. A record of endpoints, one shared monad

Once you have more than a handful of routes, naming and folding them by hand gets old — and there's a stronger reason to reach for a record than just saving keystrokes (the next section). `endpoints` builds a whole record of `Endpoint`s from a record of `Contract`s and a record of `Function`s at once, sharing one natural transformation:

```
data Routes f = Routes
    { getUser    :: f UserShape
    , createUser :: f CreateUserShape
    } deriving (Generic)

contracts :: Routes Contract
contracts = Routes { getUser = getUserContract, createUser = createUserContract }
```

```
handlers :: Routes (Function IO)
handlers = Routes { getUser = fn getUserH, createUser = fn createUserH }

myEndpoints :: Routes (Endpoint IO)
myEndpoints = endpoints id contracts handlers

app = run (handles myEndpoints) catchAll
```

`handles` collapses the record back down to the foundational `[Handle]` — the same currency as option 2, so a record-built app and a hand-built one concatenate freely (more on this below).

4. A record of endpoints, different monads per field

Sometimes routes genuinely don't share one monad — some run in plain `IO`, others in an application monad carrying config or a database connection. Declaring `Routes`'s field type as `f AppM Shape1` instead of `f Shape1` (two type arguments instead of one) lets each field name its own monad directly, the same way each field already names its own `Shape`:

```
data Routes f = Routes
    { getUser    :: f IO   UserShape
    , createUser :: f AppM CreateUserShape
    }

transforms :: Routes Transformer
transforms = Routes { getUser = Transformer id, createUser = Transformer runAppMToIO }

contracts :: Routes (Morph Contract)
contracts = Routes { getUser = morph getUserContract, createUser = morph createUserContract }

handlers :: Routes Function
handlers = Routes { getUser = fn getUserH, createUser = fn createUserH }

myEndpoints :: Routes Endpoint
myEndpoints = endpointsVia transforms contracts handlers

app = run (handles myEndpoints) catchAll
```

`Morph` lifts a plain, monad-agnostic `Contract` into the two-argument slot this convention needs; `Transformer` carries each field's own natural transformation. Argument order mirrors `endpoint`: the transform-like thing first, then contracts, then handlers. The result stays `Routes Endpoint` rather than `Routes (Endpoint IO)` — each field's monad is already fixed by the record's own field declarations, so there's nothing to normalize away, and `route`/`scope`/`handle`/`handles` don't need one uniform monad anyway.

This isn't a fallback for when option 3 "doesn't work" — it's a genuinely different capability. A record whose field type has one type argument *cannot* let fields disagree on a monad, for the same reason `Box f = Box (f Int) (f Bool)` can't let two fields disagree on which `String` fills `Either String`'s error slot once you've committed to `f = Either String` — the argument was already spent before either field got a say. Letting it vary means giving each field two arguments to work with, which is a structurally different record shape, not a tweak to the first one.

Mixing

All four interoperate rather than compete. `handles` works on the output of *either* `endpoints` or `endpointsVia`, and the result concatenates with hand-built `Handle`s exactly like any other list:

```
app = run (handles myEndpoints ++ [handle oneOffEndpoint]) catchAll
```

One declaration, several outputs

Here's the actual payoff for building `Routes` as a record in the first place, beyond convenience: the *same* `contracts :: Routes Contract` value that fed `endpoints` above can also produce a type-safe HTTP client, a type-safe link/URL builder, and an OpenAPI document — and because all three, plus the server, are built from the identical value, none of them can silently drift out of sync with what the server actually serves. A `Client` field exists *because* a matching `Contract` field exists; there is no way to construct one that doesn't correspond to a real route.

```
myClient :: Routes Client
myClient = client contracts (ClientSettings { manager = mgr, baseUrl = "https://api.example.com" })

myLinks :: Routes Link
myLinks = links contracts

myDocs :: OpenApi
myDocs = openApi contracts
```

`client`/`links`/`openApi` only ever need `record Contract` — never a `Function` or `Endpoint` — because a contract alone already contains everything a client call, a URL, or a doc entry needs to know. Building the actual server is the one job that additionally needs handlers, which is why `endpoints`/`endpointsVia` are the only two of these that ask for one.

Each of `client`/`links`/`openApi` has a heterogeneous-`n` counterpart — `clientVia`/`linksVia`/`openApiVia` — mirroring `endpointsVia` exactly, taking `record (Morph Contract)` instead of `record Contract`, for use with the same `Routes` declared with a two-argument `f`. Since none of these three ever look at `n` in the first place, "heterogeneous" doesn't mean anything different for them — it only means the *record* they're reading is shaped for `endpointsVia`'s convention instead of `endpoints`'.

Organizing routes with nested records

A large API is rarely one flat list — it has groups: user routes, post routes, admin routes. A record field can be another record of the same shape, and every function above — `endpoints`, `endpointsVia`, `client`, `links`, `openApi`, and their heterogeneous counterparts, plus `handles` — recurses into it automatically:

```
data UserRoutes f = UserRoutes
    { get    :: f UserGetShape
    , create :: f UserCreateShape
    } deriving (Generic)

data Routes f = Routes
    { users :: UserRoutes f
    } deriving (Generic)
```

The payoff is that the grouping survives into *every* generated artifact, not just the server, because it's part of the `Contracts`' own shape:

```
myClient.users.create   -- a real, directly-typed, callable client function
myLinks.users.get       -- a real, directly-typed link builder
```

This is deliberately not the same job `[Handle]` does. `Handle` erases `n`/`shape` so a list can be homogeneous — which is exactly right for the server/docs case, where the end result (`Wai.Middleware`, `OpenApi`) is one opaque value nothing downstream inspects further. But `Client`/`Link` are used *as* the typed record they

came in — flattening them to `[Handle]` would destroy exactly the structure that makes `api.users.create` mean anything, and there'd be no way to prove later "these five things are the user group." Nesting records is how that grouping stays real, all the way through, for the two artifacts where it actually matters.

Why this is worth the extra step

Everything in this document is optional machinery layered on top of two plain ideas: a `Contract` is data, and a record is just a record. `route` alone gets you a server. `Handle`/`run` gets you a server *and* docs from one list, with no `Generic` at all. `endpoints`/`endpointsVia` cost you `Generic` derivation in exchange for a record you can also hand to `client`, `links`, and `openApi` — turning "keep the client in sync with the server by hand" into a compile error if you ever fail to. Nesting records is how that guarantee stays organized once an API outgrows a single flat list of routes.

None of it is required. All of it composes with plain Haskell you already know — `(.)`, `(++)`, ordinary records, ordinary lists — because none of these functions ever invented a bespoke type to hold your API in. They hand back `Wai.Middleware`, `[Handle]`, `OpenApi`, ordinary records: things the rest of the language already knows how to combine.